Recap: Google sign-in and saved recaps

A practical guide for the capstone team · October 30, 2026

The idea

Sign in with Google before uploading. Save recaps under a permanent account. Use a cookie to remember the current session.

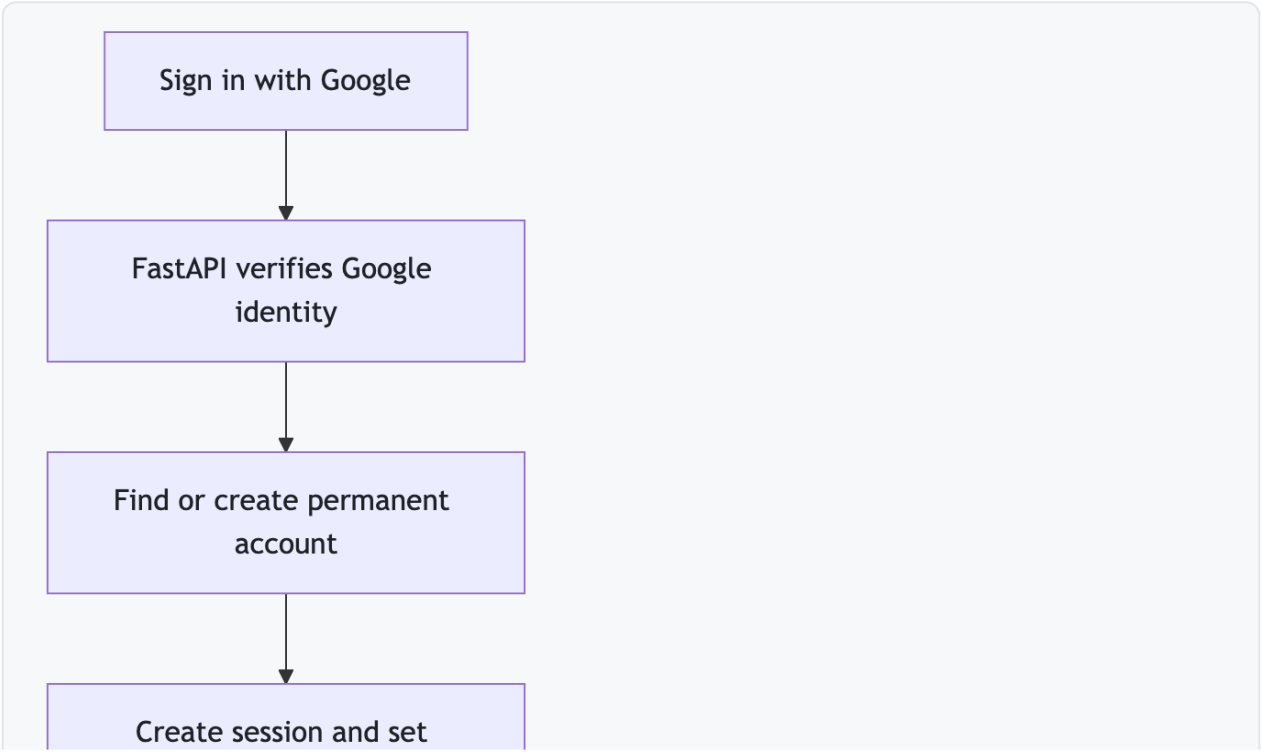
If someone clears their cookies, they sign in again with the same Google account and get their recaps back.

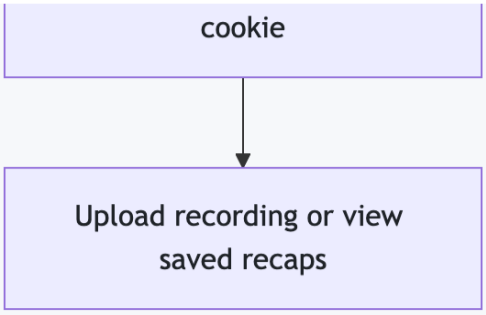
Clearing cookies does not delete database records. The problem with the original guest design is that the cookie is the only way to access those records. Google sign-in gives the user a way to reconnect to their account.

This guide updates the ownership model in database-archive.pdf. Use the original database guide for detailed transcript and summary table definitions.

1. How sign-in works

1. The user selects **Sign in with Google** and chooses an account.
2. Google returns a signed identity token.
3. FastAPI verifies the token using Google's supported Python library.
4. The backend finds or creates the user's permanent account in SQLite.
5. It creates a session and sets a cookie. The user can now upload recordings and open their recaps.





Use Google's `sub` **identifier** to recognize a returning account. Store it as `google_sub`. Do not use email as the account's permanent identifier because email can change.

Google handles sign-in; Recap still stores the meetings and decides who can access them. This does not require Firebase or moving the database to Google.

Reference: Google's server-side verification guide.

2. What changes in the database

Keep the existing `owners` table name and `meetings.owner_id` relationship. An owner now represents a permanent Google-linked account instead of one browser.

Table	Important fields	What it represents
`owners`	`id`, unique `google_sub`, optional `display_name`, `created_at`	One permanent Recap account
`sessions`	`id`, `owner_id`, unique `token_hash`, `created_at`, `last_seen_at`, `expires_at`, nullable `revoked_at`	One browser's signed-in session
`meetings`	`id`, `owner_id`, `title`, `status`, `created_at`	One uploaded meeting belonging to an account

`owners.id` is an internal database ID. Both `sessions.owner_id` and `meetings.owner_id` reference it through foreign keys.

For example:

- Alex signs in for the first time. Recap creates owner **42** and a session for that owner.
- Alex uploads a recording. The backend saves the meeting with `owner_id = 42`.
- Alex clears cookies and signs in again. The verified Google identifier still maps to owner **42**, so the same meetings appear.

The backend must choose `owner_id` from the authenticated session. The frontend must not choose it. Every meeting read, export, change, or deletion must check that it belongs to that owner. Apply this to related transcripts and summaries too.

The remaining archive tables stay as described in the original guide:

Table	Stores
`transcripts`	The meeting transcript
`transcript_segments`	Optional timestamped portions of the transcript
`summaries`	The overview and structured recap output
`decisions`	Decisions associated with a summary
`action_items`	Tasks and any identifiable owner or deadline
`outline_topics`	Topics and their supported timestamps

An action item's `owner_name` is a person mentioned in the meeting. It is separate from the account's `owner_id`.

3. What the cookie does

The cookie remembers the session, not the meetings. Give it a random secret and store only the secret's hash in `sessions`.

On each protected request, the backend resolves the cookie to a valid session, gets its `owner_id`, and accesses only that owner's records.

Event	Expected result
Clear cookies or session expires	Sign in again; saved recaps remain
Sign out	Revoke the session and clear its cookie; keep the account and recaps
Sign in on another device	Show the same archive when using the same Google account and Recap server
Sign in with another Google account	Show that account's separate archive
Delete a meeting	Deliberately remove that meeting's stored content and media

For the HTTPS demo, use `HttpOnly`, `Secure`, and appropriate `SameSite` cookie settings. Create a fresh session after sign-in, enforce expiration on the server, and use established CSRF and login nonce protections. Do not log identity tokens or session secrets. Use supported authentication/session components rather than writing these mechanisms from scratch.

References: session management and CSRF protection.

4. Setting up Google sign-in

1. Create a team-managed Google Cloud project and configure its app branding and audience.
2. Create an OAuth client of type **Web application**.
3. Register the frontend origin and the backend login URL for the chosen sign-in flow. Add demo accounts if the app is restricted to test users.
4. Add the Google sign-in button to React. For a straightforward first version, use Google's redirect flow with FastAPI handling the credential POST at `/auth/google`.
5. Configure the backend to verify tokens for that client ID, including signature, audience, issuer, and expiration. Follow the integration's CSRF and nonce requirements.

The client ID is public configuration. Keep server secrets and LLM API keys out of the frontend and source control.

Request only basic identity information. Drive and Calendar access are separate features and can wait. Leave One Tap and automatic sign-in out of the first version.

References: Google setup and sign-in button integration.

5. Connecting sign-in to the existing app

The recording pipeline stays the same:

Signed-in upload → audio processing → transcription → structured recap → save to SQLite → display recap.

The main difference is that every new meeting receives the permanent account's `owner_id`.

These are the main routes the team needs. The chosen authentication component may also require supporting routes for CSRF or login state.

Route	Purpose
`POST /auth/google`	Verify Google sign-in, find/create the account, and start a session
`GET /auth/me`	Tell React whether the user is signed in
`POST /auth/logout`	End the current session without deleting data
`POST /meetings`	Upload a recording for the signed-in account
`GET /meetings`	List that account's meetings
`GET /meetings/{id}`	Return an owned meeting's status and recap

Validate uploads on the backend. Save a validated summary and its related rows in one database transaction. Remove temporary recordings after processing, including cleanup for

database transaction. Remove temporary recordings after processing, including cleanup for failed jobs. Keep transcripts and recaps for later access. Store media outside the public website directory and keep meeting content out of logs.

The original archive guide covers status values, exports, and the detailed storage design.

6. Keeping the data available

Google sign-in restores access to an account. **Persistent storage and backups protect the data itself.**

Keep `recap.db` in persistent storage so restarting the app or rebuilding a Docker container does not erase it. Use SQLite's supported backup method for a running database: SQLite backup guidance.

Another device can access the same recaps only when it connects to the same Recap deployment. Two independently running local installations still have separate databases.

If guest records already exist, preserve them during migration. Linking them to Google requires proof of access to the guest archive and the Google account. A lost guest cookie cannot be recovered simply by matching a name or email. If no guest database exists yet, start directly with Google-linked accounts.

7. What to build first

1. Add permanent accounts and separate sessions to SQLite.
2. Connect Google sign-in and sign-out.
3. Make meeting routes use the authenticated owner.
4. Connect the upload, transcription, and summarization pieces.
5. Add the saved-recap library.

For December, focus on those pieces and understandable error messages. Add Markdown export if time permits. Search, PDF polish, summary version history, speaker identification, and Google integrations can wait until spring.

The behavior to aim for is simple: **upload a recording, clear cookies, sign in again, and reopen the same recap.**